LLMsVerifier

LLMsVerifier

Verify. Monitor. Optimize.

Summary

LLMsVerifier is an enterprise-grade platform for verifying, monitoring, and optimizing Large Language Models across many providers, built on a mandatory “Do you see my code?” verification test so that only models proven to actually work are ever marked usable or exported.

Short description

A Go platform that verifies, benchmarks, monitors, and optimizes LLMs across multiple providers. Every model must pass a mandatory code-visibility test before use; it then runs latency, streaming, function-calling, vision, and embedding checks, and exports verified-only configurations for AI CLI tools.

Long description

LLMsVerifier is a comprehensive platform for verifying, monitoring, and optimizing LLM performance across multiple providers. Its core principle is *mandatory verification*, and it is uncompromising about it: before any model is marked usable — or allowed into an exported configuration — it must affirmatively pass a “Do you see my code?” test that makes real HTTP calls to the provider and analyzes the response for genuine comprehension, not a plausible-looking echo. A model that can’t demonstrably see and understand your input simply never earns the “usable” flag. Beyond that gate, the Verifier Engine runs a full battery of capability tests — existence, responsiveness, latency,

streaming, function calling, vision, embeddings — and a Reporter Engine turns the results into markdown and JSON reports you can act on.

The system is modular and event-driven, exposing CLI, TUI, Web, and REST API interfaces over a core of Verifier Engine, Reporter Engine, and Configuration Manager, and it doesn't stop at verification. Advanced layers add a Supervisor/Worker pattern for LLM-powered task decomposition, sliding-window plus LLM-summarization context management so very long sessions don't fall off a cliff, cloud-backed checkpointing, and a failover system with circuit breakers and latency-based routing. The surrounding infrastructure is production-shaped: a pub/sub event bus, cron scheduling, pricing/limits detection, a vector database for RAG, and an export system. A signature branding convention appends an `(llmsvd)` suffix to every generated provider/model, so a verified output is traceable at a glance and can never be confused with an unvetted one — and only verified models are ever written into exported configs for AI CLI tools such as OpenCode, Crush, and Claude Code. It ships with the operational tooling teams actually need in production: Docker/Kubernetes/Helm deployment, Prometheus/Grafana monitoring, LDAP/SSO, and SQLCipher-encrypted storage.

Why we built it

Because configuration-only checking is unreliable — an API key can expire, a model can be deprecated, and a config file tells you nothing about real latency, real errors, or whether the model can actually see and understand your input. LLMsVerifier replaces "it's in the config, so it must work" with proof: only models that demonstrably respond correctly are marked usable and exported.

Why it's a game-changer

It makes LLM fleets *trustworthy* — a word rarely earned in a space full of configs that lie by omission. Instead of hoping a configured model works, teams get an enforced, testable guarantee that every model in play has passed real verification, with monitoring, failover, and verified-only export closing the loop from proof to production. Within the Helix ecosystem it becomes the single source of truth for LLM model, provider, and verification metadata: other services (HelixTranslate among them) route against it, so an entire platform inherits one honest answer to "which models actually work right now?" instead of each team maintaining its own hopeful guess.

What’s innovative

- **Mandatory “Do you see my code?” verification** — a real, HTTP-backed comprehension gate a model must pass before it is ever usable; the product’s signature differentiator and the reason nothing unproven slips through.
- **Verified-only configuration export** — generated configs for AI CLI tools contain *only* models that passed verification, so the config you ship can’t quietly reintroduce a broken model.
- **`(llmsvd)` branding-suffix system** — every generated provider/model carries a traceable suffix, making verified provenance visible everywhere the output travels.
- **Capability detection** across many CLI agents and providers — it fingerprints streaming types (SSE, WebSocket, JSONL, EventStream), compression, and caching behaviors rather than assuming them.
- **Resilient failover** — circuit breakers, latency-based routing that reroutes when time-to-first-token crosses a threshold, health probes, and weighted traffic split keep a fleet responsive when individual providers wobble.
- **Long-running autonomy** — a Supervisor/Worker decomposition pattern plus checkpointing and memory integration sustain extended sessions that would otherwise exhaust context.
- **RAG / vector-DB integration** for grounded context enhancement.

Biggest technical challenges & how we solved them

- **Proving a model actually works, not just that it’s configured.** The whole point, and the hardest part. Solved by the mandatory code-visibility test that makes real API calls and analyzes the responses for affirmative comprehension, backed by a broad suite of capability tests — and then by refusing to export anything that didn’t pass, so proof, not configuration, gates production.
- **Reliability across many flaky third-party providers.** Solved with a failover orchestrator that treats provider instability as the normal case: circuit breakers mark a provider degraded after N failures in M seconds, latency-based routing steers away from slow endpoints, periodic health checks probe for recovery, and weighted routing balances cost-effective against premium models.
- **Sustaining very long, autonomous sessions.** Solved with the Supervisor/Worker decomposition pattern that breaks big work into tractable pieces, periodic checkpointing to cloud storage so progress survives interruption, and layered context

management (sliding window + LLM summarization + RAG) so the model keeps the thread without drowning in tokens.
- **Provider sprawl.** Solved by hiding many per-provider Go adapters behind one common interface, with the real endpoints enumerated centrally — so adding a provider is a contained change, not a ripple through the codebase.

Tech stack

- **Go** — chosen as the core platform language for its concurrency; it drives a multi-threaded Verifier Engine that can probe many models in parallel, plus the surrounding services.
- **Gin** — chosen as the REST API server, carrying JWT auth, rate limiting, and WebSocket/SSE endpoints.
- **SQLite + SQLCipher** — chosen for embedded storage with database-level encryption, because verification data (keys, results) is sensitive and should be encrypted at rest by default.
- **Redis** — chosen as the caching layer to keep hot verification and metadata lookups fast.
- **RabbitMQ + Kafka** — chosen to power the event-driven architecture: messaging and streaming that decouple producers from consumers across the platform.
- **gRPC + Protocol Buffers** — chosen for strongly-typed inter-service communication and event transport between components.
- **QUIC / HTTP-3 (quic-go)** — chosen for modern transport support (repo docs flag HTTP/3 provider availability as limited — a capability offered, not a universal claim).
- **JWT + LDAP/NTLM** — chosen for enterprise authentication so the platform slots into existing corporate identity (SSO/SAML/OIDC claimed in docs).
- **Viper (config), Logrus (logging), Brotli/compress (compression)** — the operational plumbing: flexible configuration, structured logs, and payload compression.
- **Angular** — chosen for the Web single-page application, the visual front door to verification and monitoring.
- **Python + JavaScript SDKs** — chosen to give client teams first-class access, documented via OpenAPI/Swagger.
- **Docker, Kubernetes, Helm** — chosen for production deployment with health monitoring and autoscaling, so a verification fleet scales like any modern service.
- **Prometheus + Grafana** — chosen for metrics and dashboards, making the platform’s own health as observable as the models it watches.
- **Testify (Go) + node –test/jsdom (web)** — chosen for layered testing across the Go core and the web front end.

Status & honesty notes

- **Status: beta.** The Go source implements real HTTP verification (one legacy doc that describes verification as config-only is aspirational and outdated — the code is authoritative).
- **License: TBD.** The README states MIT while a Dockerfile label states Apache-2.0 — resolve before publishing.
- Provider count: the README says “12 adapters” but the providers directory lists roughly 26 — treat as “12+ / more in progress.” Numerous aspirational “FINAL/COMPLETE” status files exist; code, docs, and `go.mod` are authoritative.
- The repository lives in the `vasic-digital` org, but functionally it is the trust layer of the Helix LLM-infrastructure cluster.

Priority tier: Helix-primary (LLM-infrastructure cluster; single source of truth for LLM/provider/verification metadata). Ranks after HelixTrack.